COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 2
DATABASE DESIGN:
ENTITY-RELATIONSHIP (E-R) DATA MODEL

DATABASE DESIGN: OUTLINE

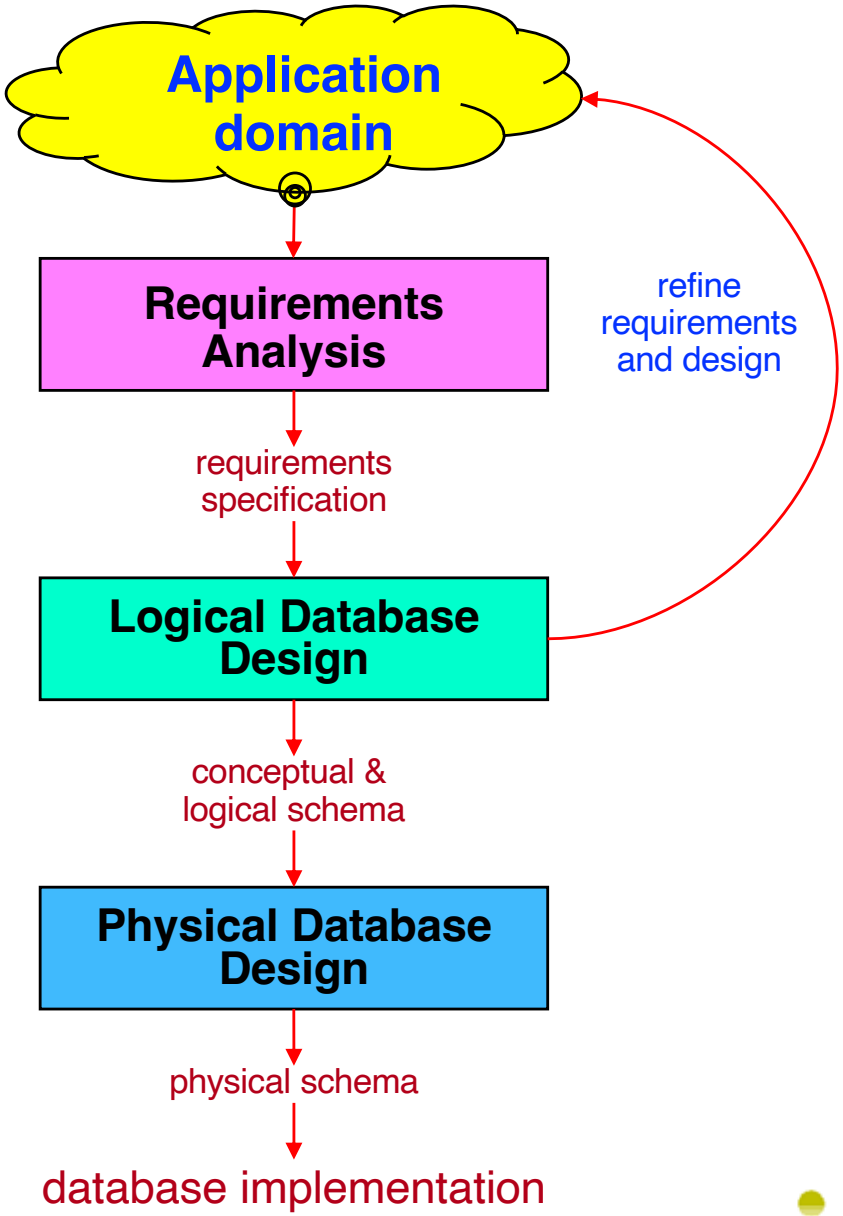
Database Design Process

Entity-Relationship (E-R) Data Model — Data Structure Types

Entity-Relationship (E-R) Data Model — Constraints

Design Choices

DATABASE DESIGN PROCESS



Database Design Goals

1. Meet the data content requirements of users.
2. Provide a natural and easy-to-understand structuring of data.
3. Support data processing requirements and any performance objectives (e.g., response time, processing time, storage space, etc.).

DATABASE DESIGN PROCESS (CONT'D)

Requirements Analysis produces a requirements specification

- Requirements analysis understands the application domain and identifies the data requirements of the application.

Logical Database Design produces one or more schemas

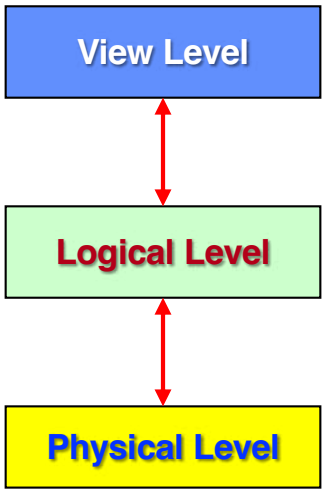
- Logical database design describes how to represent the data requirements of an application in a database and often proceeds in two phases producing two schemas.
  a) The conceptual schema describes the data requirements for a database using a DBMS-independent data model (e.g., the E-R data model).
  b) For DBMSs that use schemas, the logical schema describes the data in a database using the data definition language (DDL) of the target DBMS (e.g., SQL DDL).
     For DBMSs that do not use schemas, there is no logical schema.

Physical Database Design produces a physical schema

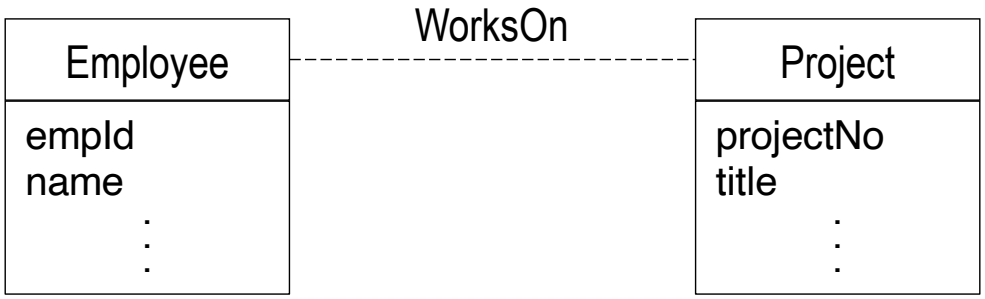
- Physical database design describes how the logical schema is stored on some storage media (e.g., data types, keys, indexing options, etc.).

ENTITY-RELATIONSHIP (E-R) MODEL

The entity-relationship (E-R) data model is used at the logical level to describe a database's overall structure.



- The E-R data model employs three basic concepts to describe data.
  1. entity (something about which we want to keep data).
  2. attribute (properties of entities—what data to keep).
  3. relationship (among entities).



Why E-R data model?
- expressiveness
- user communication
- DBMS independent

☞ **These concepts are used to describe the data requirements for an application using an entity-relationship diagram (E-R diagram).**

ENTITY

An *entity (type)* describes a set of entity instances with common:
– properties – relationships – semantics

Something in the application domain about which we want to store data.

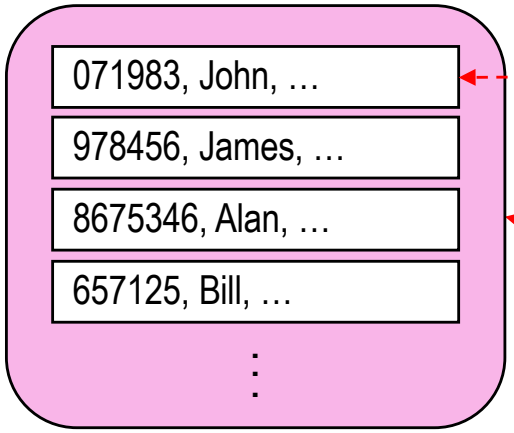
(E.g., employee, student, course, product, order, … .)

schema

Employee ← **entity (type)** (a common *description* for all employees)

☞ **Entity names must be unique in an ERD.**

data



entity (instance) (*data* about a specific employee)

entity set (the *collection* of all employee data)

An entity instance

has identity.

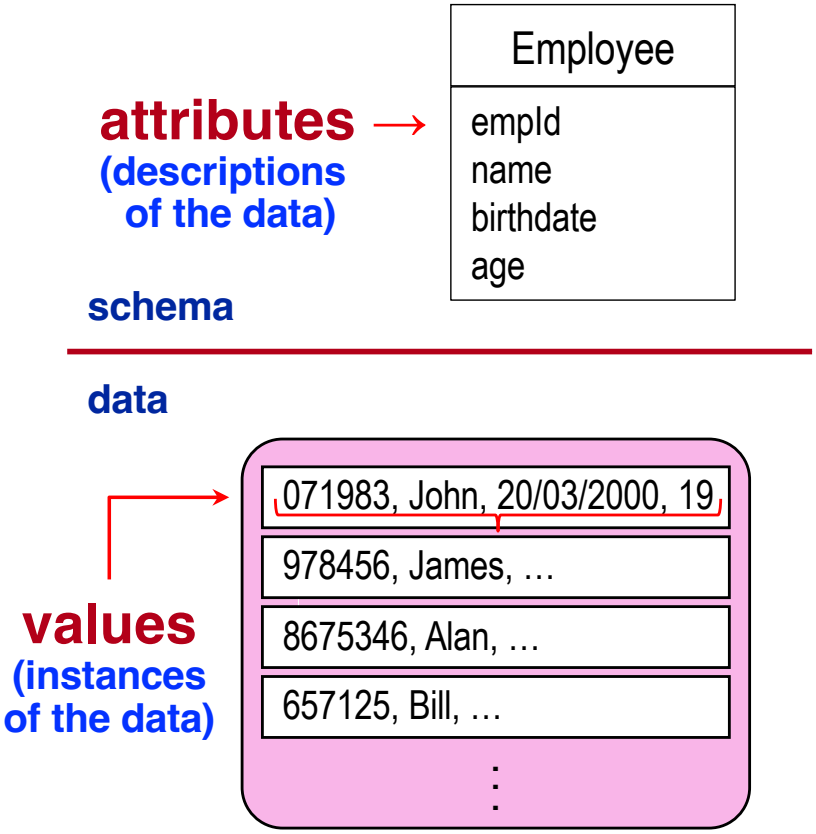
- It can be distinguished from other entity instances.

represents some real-world thing.

- It has meaning in the application domain.

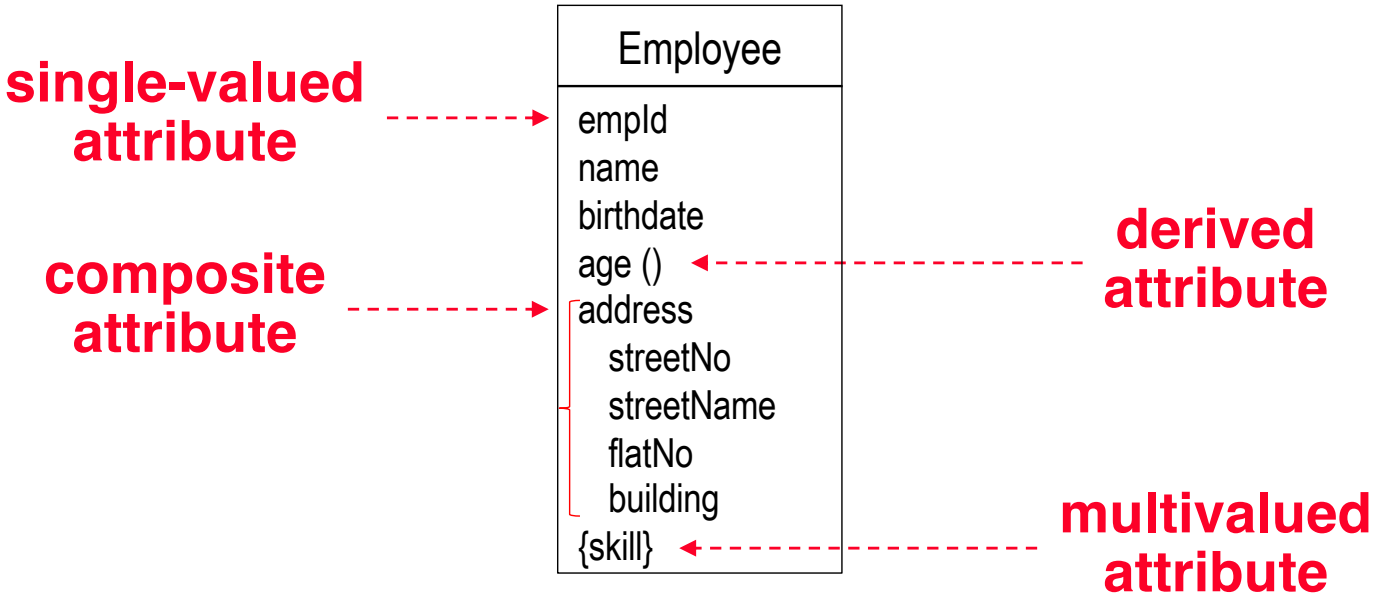
ATTRIBUTE

An *attribute* is a property of an entity and describes the data values of that property.



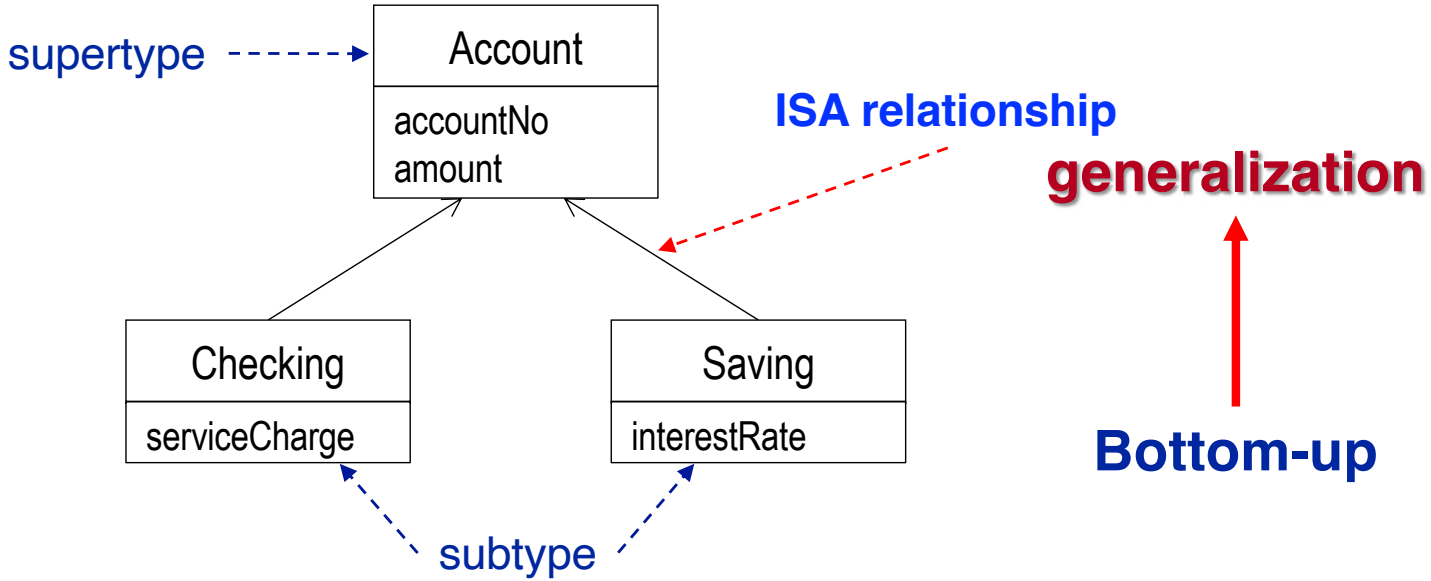
- Each attribute has a name that is unique within an entity (but not across entities).
- Most attribute values are physically stored (base attribute);
  an attribute may be calculated using stored values (derived attribute).
- An attribute value may be null
  (a special value indicating that the value is missing, unknown or not applicable).

ATTRIBUTE TYPES AND NOTATION



ENTITY GENERALIZATION/SPECIALIZATION

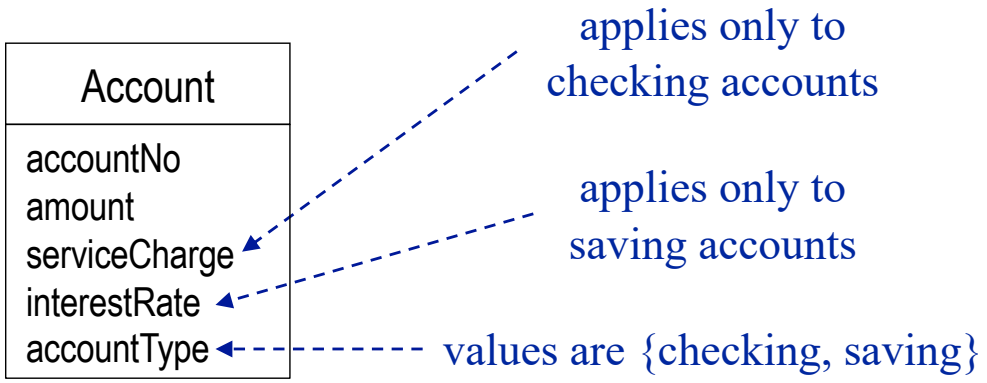
Generalization/specialization **is a relationship between the same kind of entities playing different roles.**



☞ In this example, subtype membership is user-defined (i.e., it is determined by the schema designer and not based on the value of an attribute).

ENTITY GENERALIZATION/SPECIALIZATION (CONT'D)

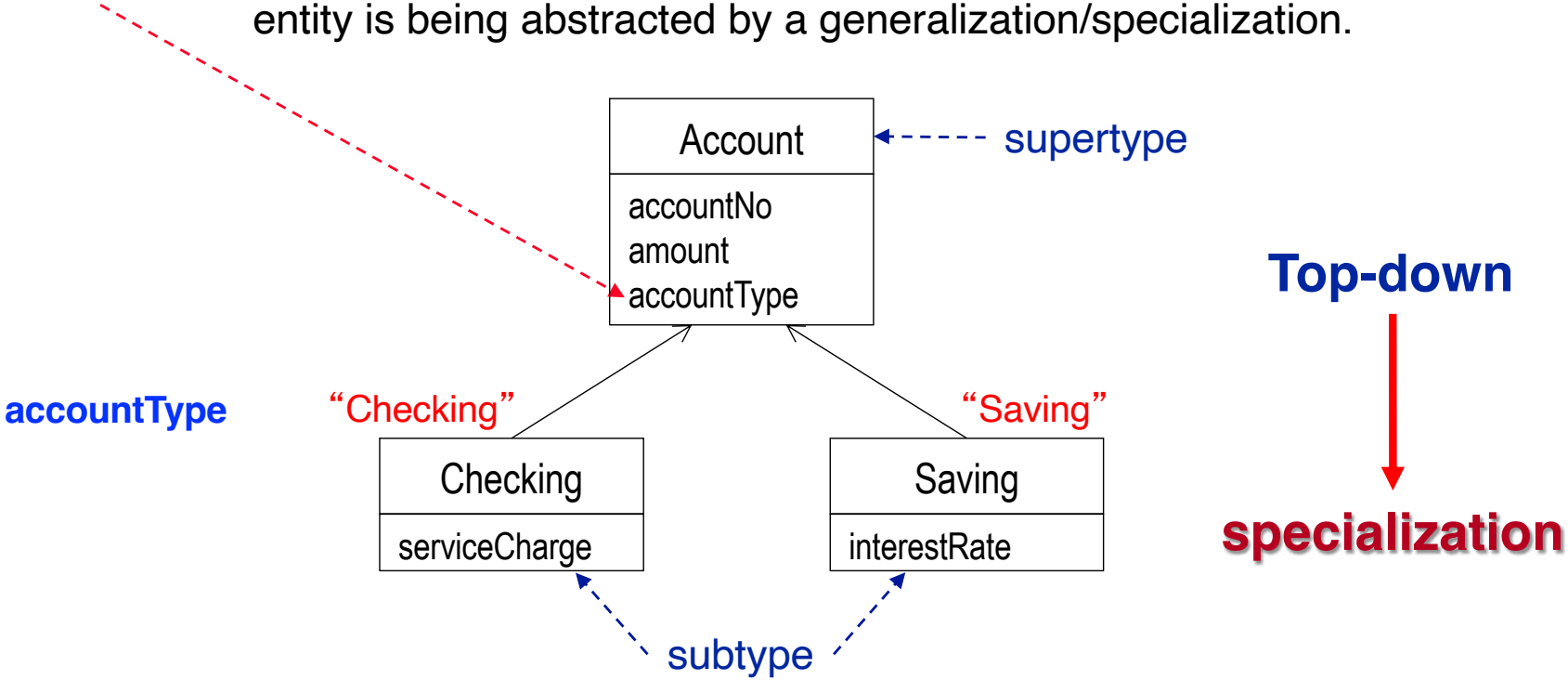
Can also be applied top-down.



ENTITY GENERALIZATION/SPECIALIZATION (CONT'D)

Can also be applied top-down.

discriminator: An attribute of enumeration type that indicates which property of an entity is being abstracted by a generalization/specialization.



☞ In this example, subtype membership is determined by a predicate (i.e., by a logical condition) on an attribute (i.e., the discriminator attribute) of the supertype.

INHERITANCE

***Inheritance* is the taking up of properties by a subtype from its supertype.**

- We extract the *common* attributes and relationships, associate them with the supertype and inherit them to the subtype(s).
  - ✓ Reduces redundancy of descriptions.
  - ✓ Promotes reusability of descriptions.
  - ✓ Simplifies modification of descriptions.

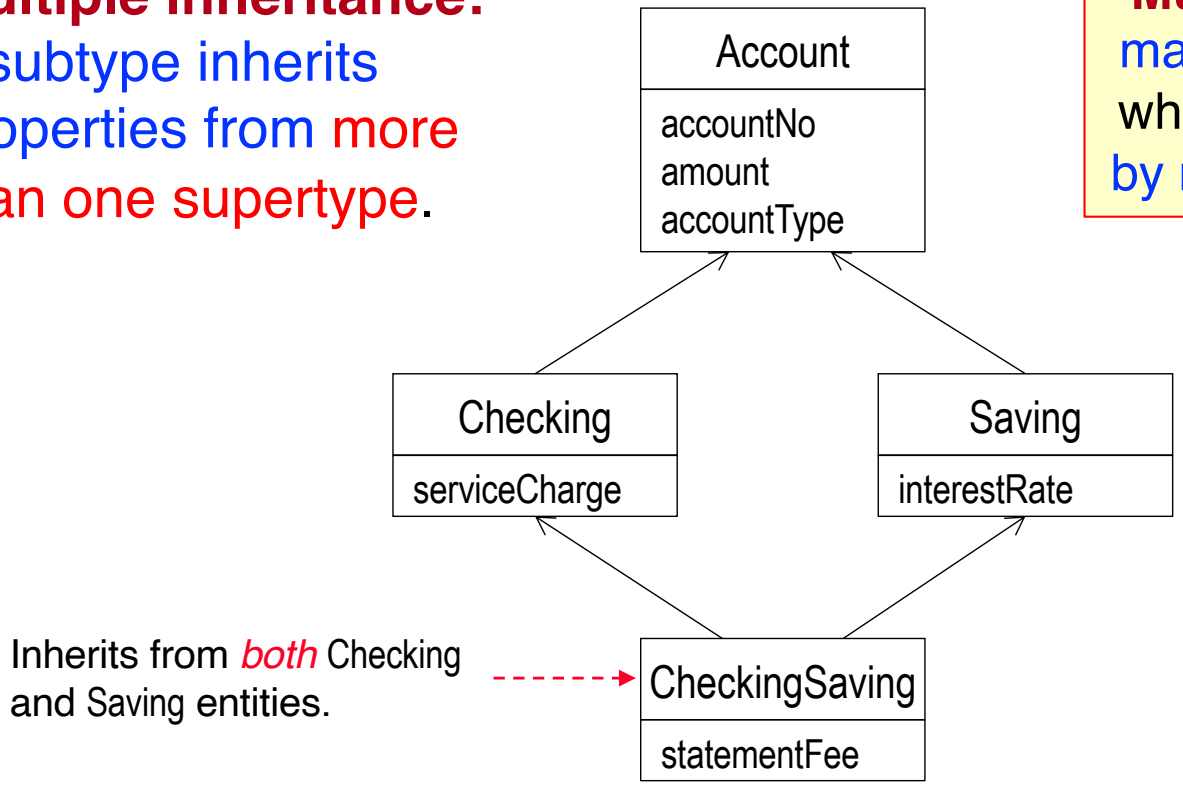
☞ **We need only define each property of an entity in one place.**

- A subtype may add new properties (attributes, relationships).

Design Guideline: Inheritance should not exceed 2-3 levels.

SINGLE VS. MULTIPLE INHERITANCE

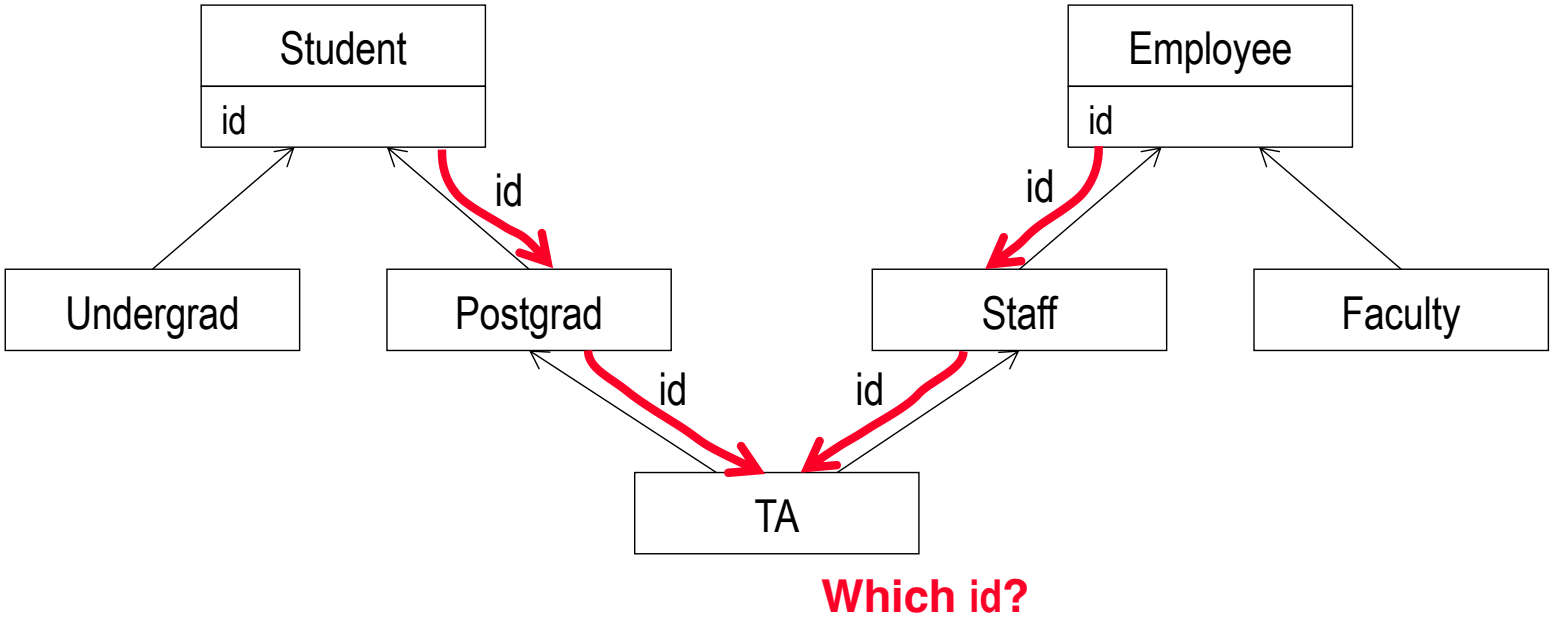
Multiple inheritance:
a subtype inherits properties from more than one supertype.



Multiple inheritance may result in conflicts, which can be resolved by renaming attributes.

For multiple inheritance, a property *from the same ancestor entity* found along more than one path is *inherited only once*.

MULTIPLE INHERITANCE EXAMPLE



Attributes should be assigned *meaningful* and *unambiguous* names.

RELATIONSHIP

A *relationship (type)* is a description of a set of relationships with common properties and semantics.

E-R diagram (ERD)

relationship (type)
(a common *description* for a relationship set)

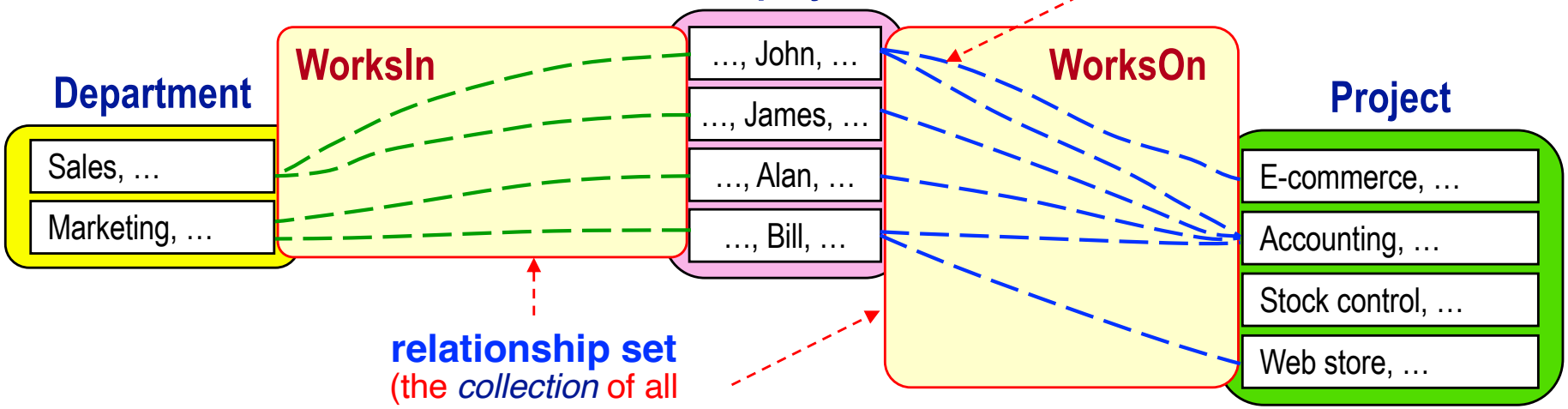
☞ **Relationship names should be unique in an ERD.**

schema

---

data

☞ **Conceptually, relationships are inherently bi-directional.**



relationship (instance)
(a specific relationship)

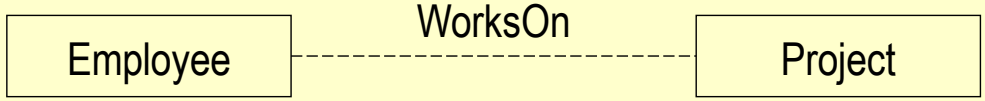
relationship set
(the *collection* of all relationships of one type)

RELATIONSHIP DEGREE

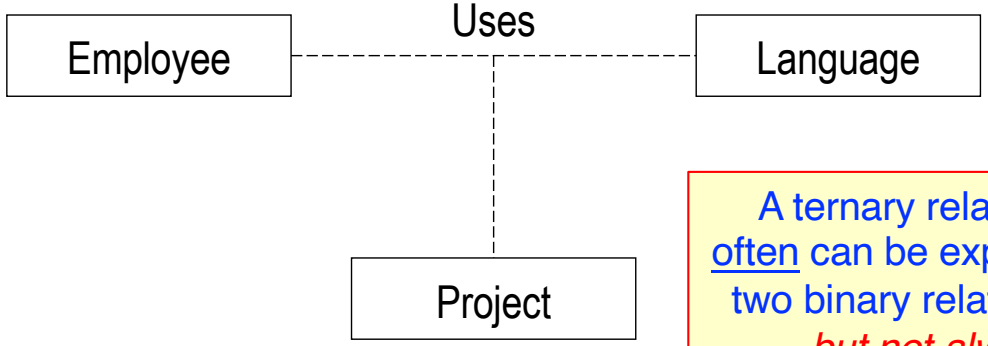
- The number of entities that participate in a relationship.

unary (reflexive) – relates *the same* entity (to itself)

binary – relates *two* different entities



ternary – relates *three* different entities



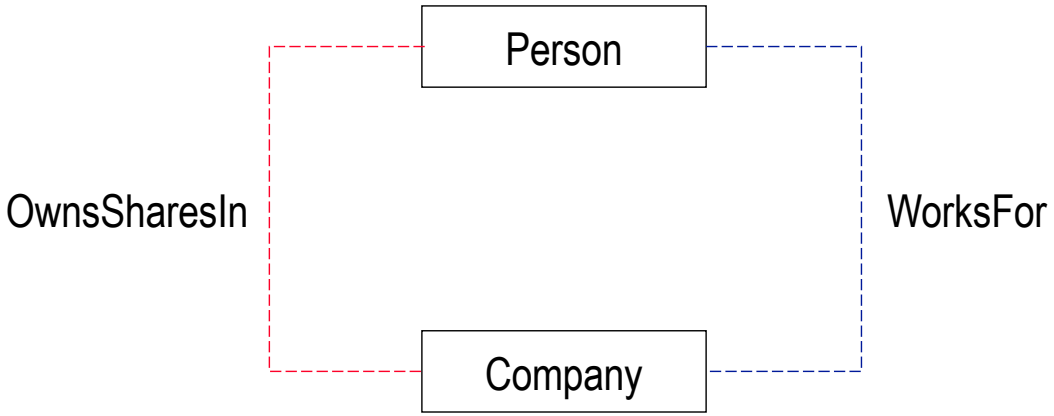
Higher degrees, though possible, are extremely rare!

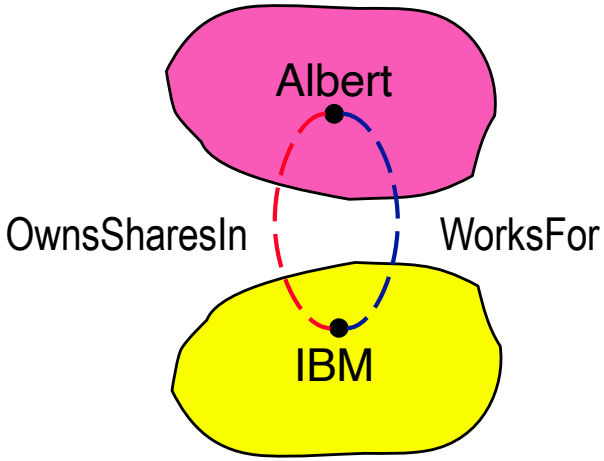
A ternary relationship often can be expressed as two binary relationships, *but not always*.

In practice, most relationships are binary.
(We will use only unary or binary relationships in this course.)

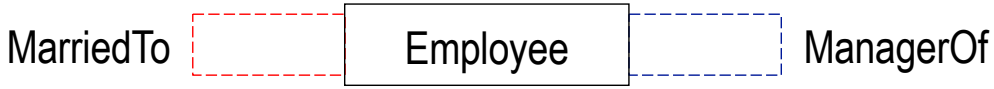
RELATIONSHIP EXAMPLES

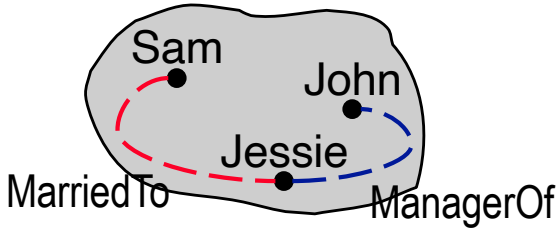
There can be more than one relationship between two entities.





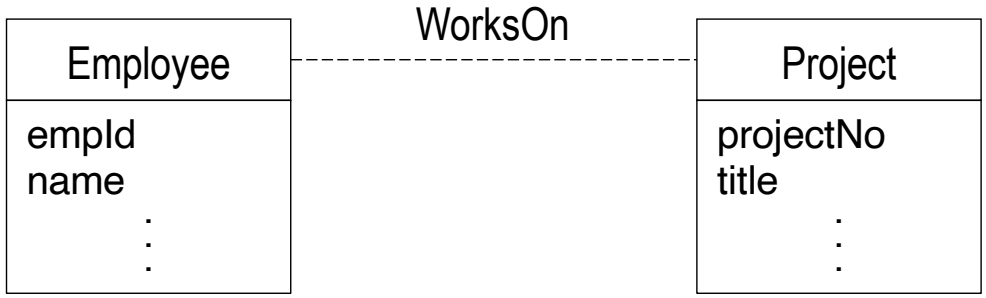
or even the same entity.

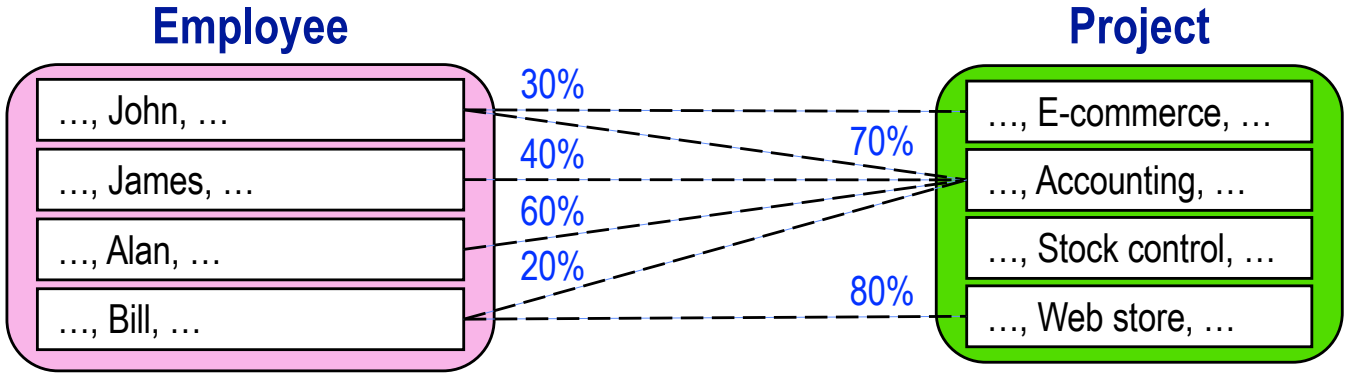




RELATIONSHIP ATTRIBUTES

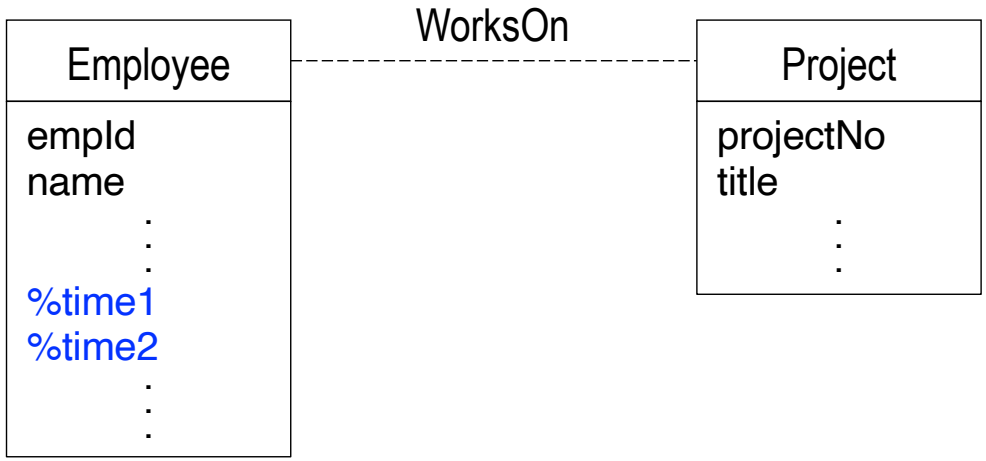
- We want to represent the percentage time that an employee works on a project.

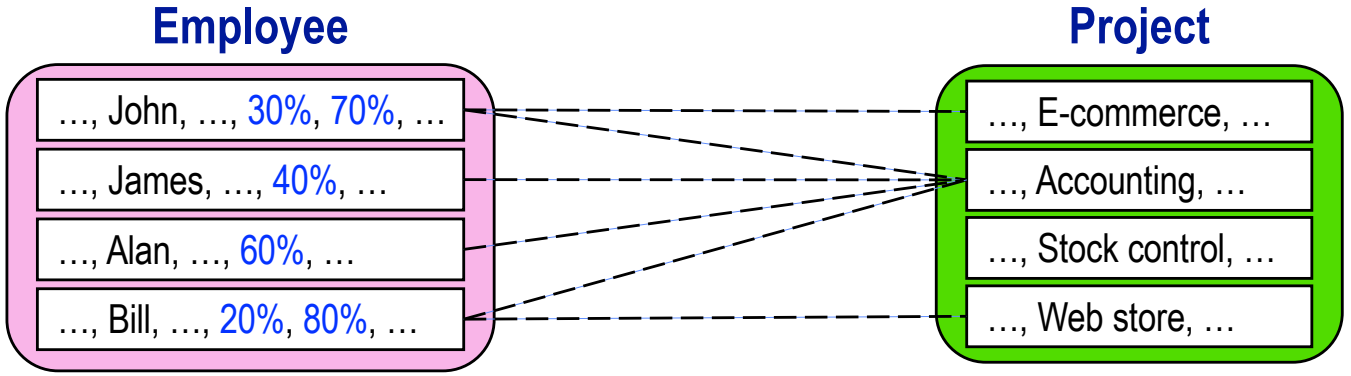




RELATIONSHIP ATTRIBUTES (CONTD)

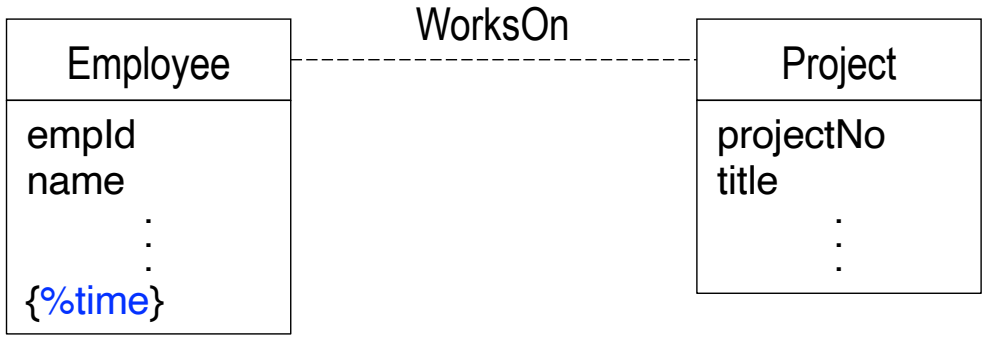
Option 1: Use many attributes (e.g., in Employee). **Is this, OK?**

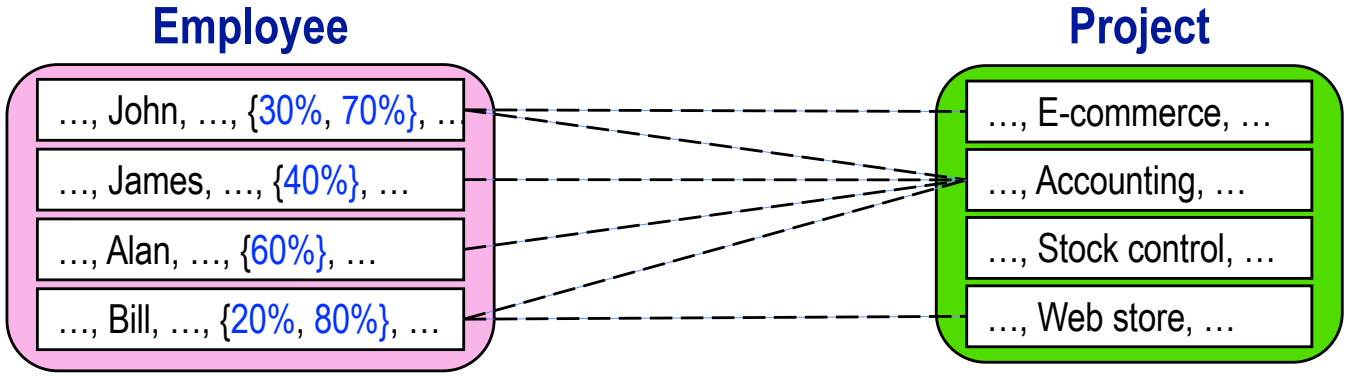




RELATIONSHIP ATTRIBUTES (CONTD)

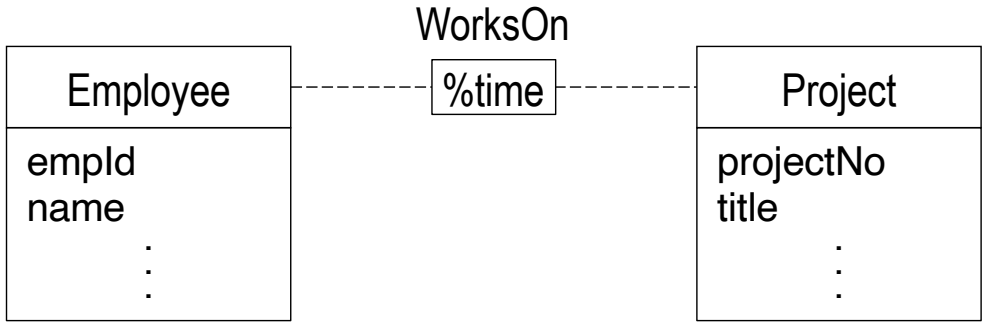
Option 2: Use a multivalued attribute (e.g., in Employee). **Is this, OK?**

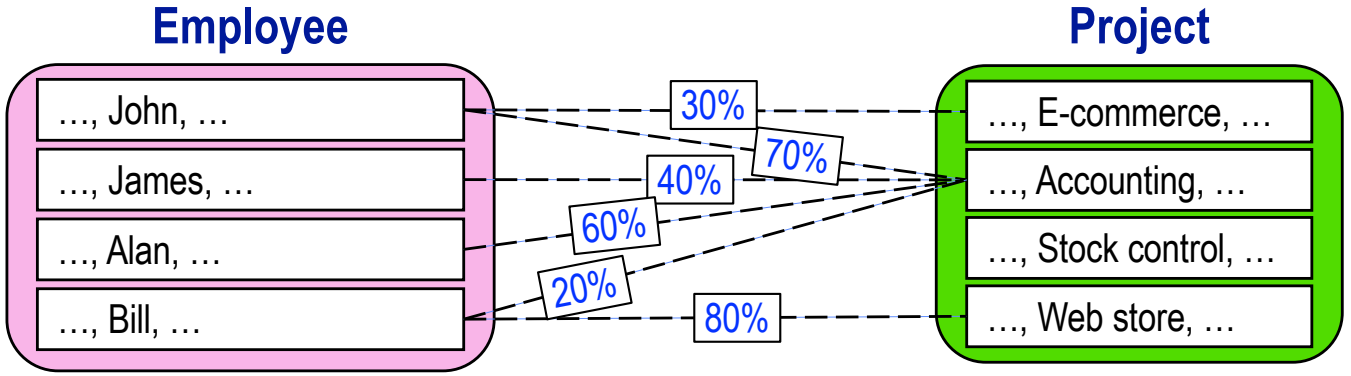




RELATIONSHIP ATTRIBUTES (CONTD)

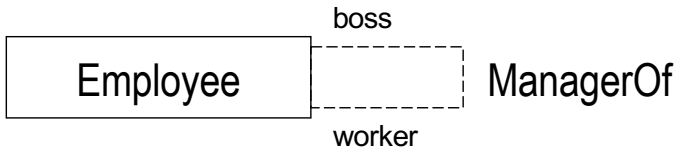
Option 3: Allow relationships to have attributes. **Is this, OK?**

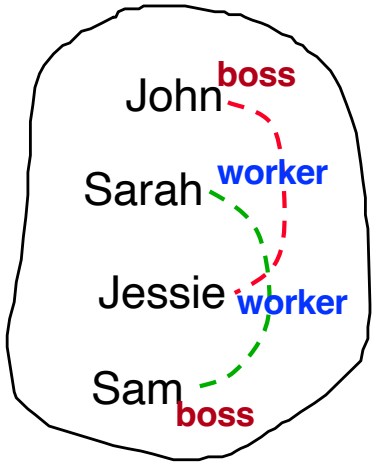




RELATIONSHIP ROLE NAMES

A *role name* is assigned to one end of a relationship to identify the role that the entity at that end plays in the relationship.





Who is the boss and who is the worker?

A role name disambiguates the role that an entity plays in a relationship.

It is necessary to use role names for unary relationships (i.e., when a relationship relates instances from the same entity).

ANALYZING APPLICATION DATA REQUIREMENTS

1. **Identify entities**
   - What are the major concepts about which data needs to be permanently stored?
   - Focus on the "big picture", not the details.
     - E.g., student, course not name, address, email, description, credits, etc.
2. **Identify relationships between entities**
   - How are the major concepts related? How do they interact?
   - What relationships/interactions need to be permanently stored.
     - E.g., students enroll in courses not students browse courses
3. **Identify properties of entities and relationships**
   - For each entity and relationship, what information needs to be permanently stored.